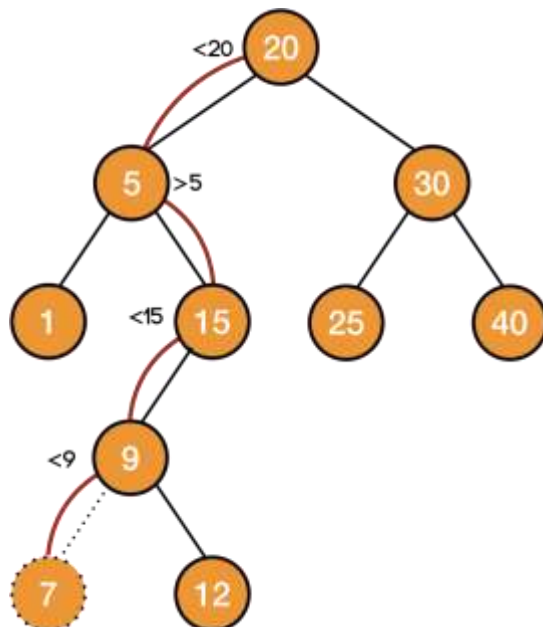


Lecture: 11
Binary Search Trees
(Part-II)

Insert Operation:

Inserting a value in the correct position is similar to searching because we try to maintain the rule that the left sub-tree is lesser than root and the right sub-tree is larger than root.

To insert an element, **we first search for that element and if the element is not found, then we insert it.** i.e. we keep going to either right sub-tree or left sub-tree depending on the value and when we reach a point left or right sub-tree is null, we put the new node there.



INSERT 7
STEP 1: 7 IS SMALLER THAN 20: GO LEFT
STEP 2: 7 IS GREATER THAN 5: GO RIGHT
STEP 3: 7 IS SMALLER THAN 15: GO LEFT
STEP 4: 7 IS SMALLER THAN 9: GO LEFT
STEP 5: NO LEFT CHILD: INSERT 7

Algorithm:

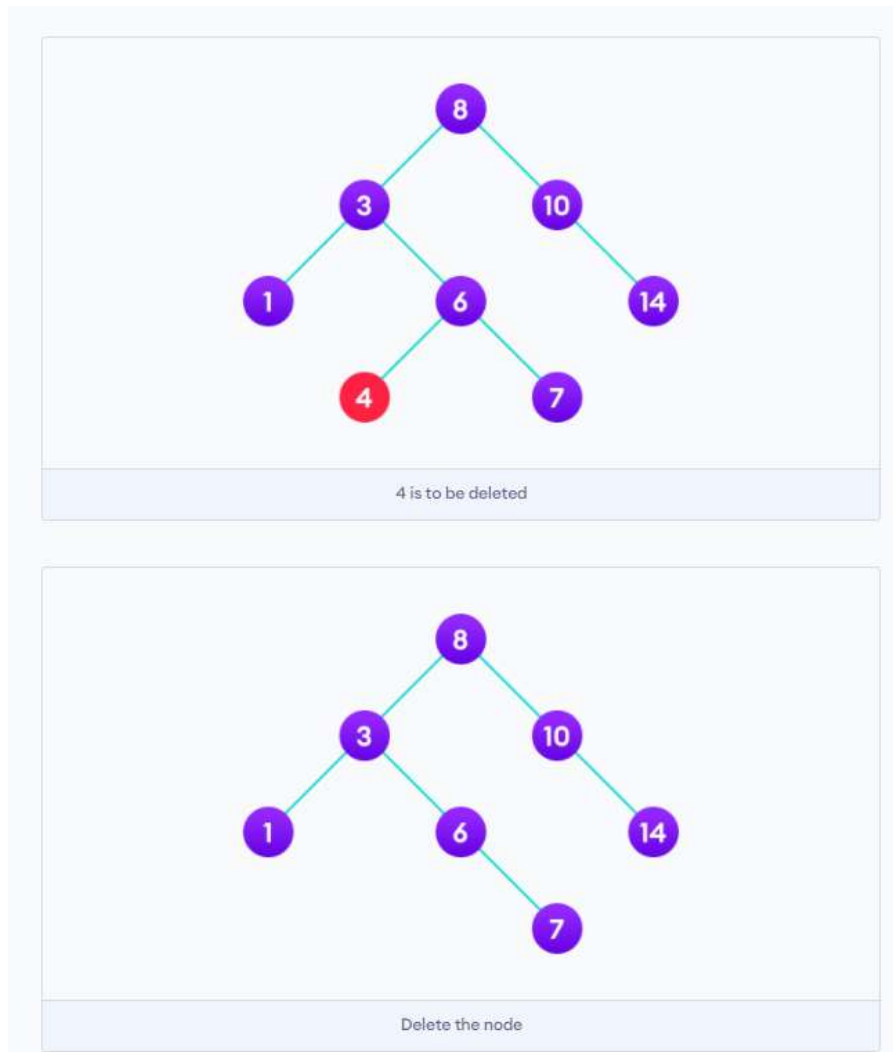
```
If node == NULL
    return createNode(data)
if (data < node->data)
    node->left = insert(node->left, data);
else if (data > node->data)
    node->right = insert(node->right, data);
return node
```

Deletion Operation:

There are three cases for deleting a node from a binary search tree:

Case:1

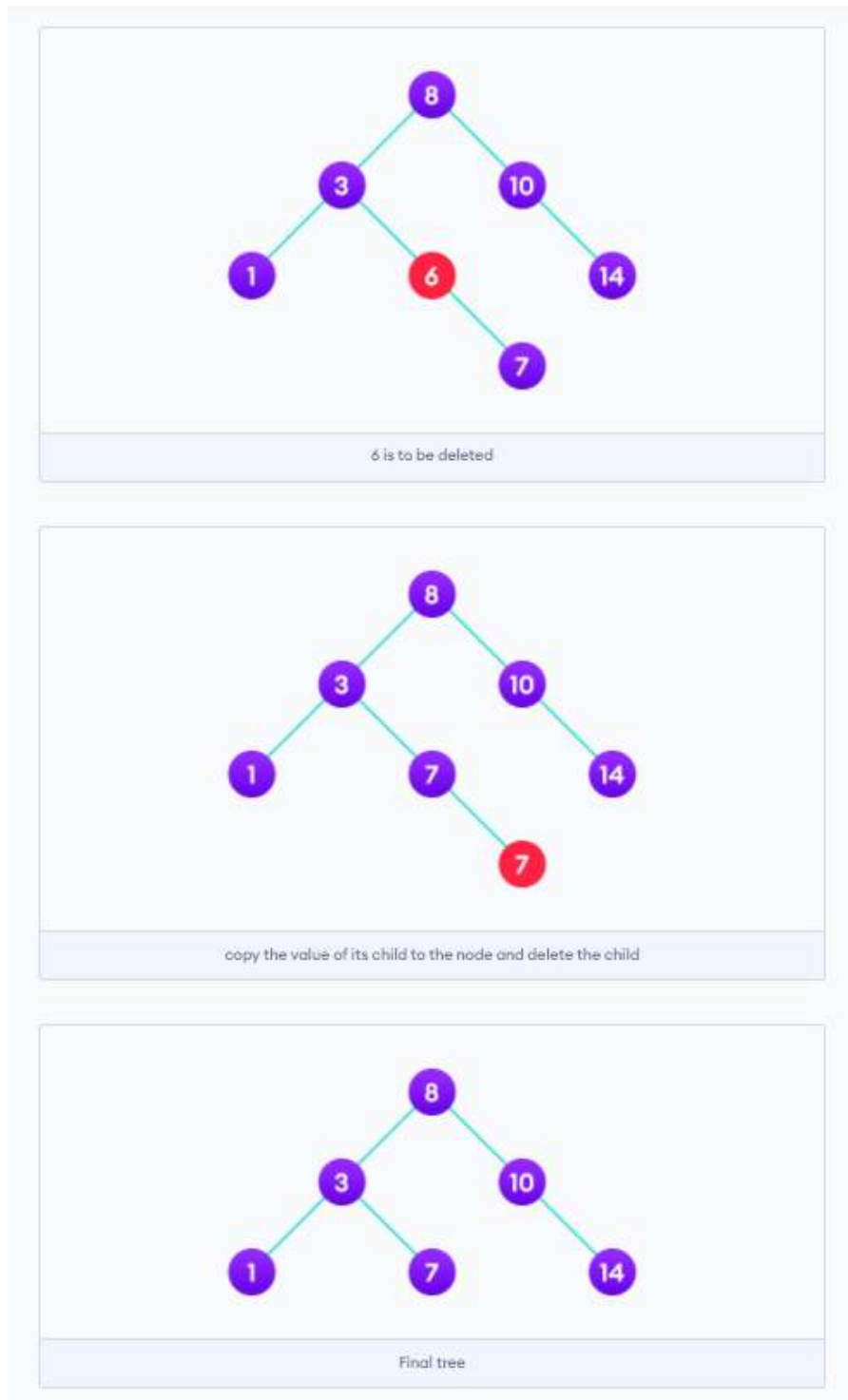
In the first case, the node to be deleted is the leaf node. In such a case, simply delete the node from the tree.



Case:2

In the second case, the node to be deleted has a single child node. In such a case follow the steps below:

1. Replace that node with its child node.
2. Remove the child node from its original position.



Case: 3

In the third case, the node to be deleted has two children. In such a case follow the steps below:

1. Get the inorder successor of that node.

2. Replace the node with the inorder successor.
3. Remove the inorder successor from its original position.

Inorder successor of N can be found by moving to the right child of N and then moving repeatedly to the left until meeting a node with an empty left subtree.

